

Tópicos de Minería de Datos - Teoría

Preparación de datos para Data Mining

Entendiendo los datos

Hay dos aspectos importantes a considerar para entender los datos con los que estamos trabajando:

- Ver su **relevancia** para el problema. Esto conlleva también analizar si hay otros datos relevantes disponibles, qué período de tiempo cubren los datos y quién es el experto en esos datos.
- Ver la **calidad** de los datos:
 - Analizar si el **número de entradas** es suficiente para que los resultados sean confiables.
 - Ver el **número de variables**
 - Si son demasiadas, se puede usar selección o extracción de variables.
 - *Rule of thumb (regla clásica)*: Se quiere tener 10 o más datos por cada variable.
 - También se tiene que ver el desbalance entre las clases. Si es muy desbalanceado se puede intentar corregir.

Limpieza de datos

- **Metadatos**: nos dan información relevante sobre los datos, tales como:
 - Tipo de variable.
 - Uso de la variable: entradas de los modelos, output, variables que dan peso estadístico, etc.
- **Formato de los datos**: por lo general es necesario convertir los datos a un formato estándar con campos numéricos. Algunos aspectos a analizar son:
 - Identificar los **datos faltantes** y cómo están simbolizados. Si el modelo no se aguanta valores faltantes, podemos: ignorar entradas con datos faltantes (si las líneas con datos faltantes son pocas), ignorar las variables si son poco útiles y tienen mucha proporción de faltantes, tratar a los faltantes con un valor particular (en cuyo caso hay que ver qué valor darle), o usar *imputation*, que es llenarlo con un valor no extremo, como la mediana o media, o incluso también podemos tratar de predecir el valor particular adecuado con un método de ML.
 - Hay que pensar a las **fechas en un formato uniforme y coherente**. Se puede usar Unix system date (número de segundos desde una fecha dada) pero eso es más confuso de seguir y propenso a errores. Otra opción es KSP, que usa el año más la fracción del año que lleva como valor. Este último formato preserva intervalos razonablemente y es más fácil de analizar.
 - Las **variables categóricas deberían pasarse a numéricas**. Para variables binarias, pueden convertirse a 0 y 1, o -1 y 1. Los datos ordinales (clasificaciones discretas pero que mantienen un orden) se debería convertir a números que preserven el orden natural y la escala (en caso de que estén escalados)
 - Las variables **nominales** (discretas sin orden) con pocos valores se pueden convertir en N variables con todas las clasificaciones en las que una sola está en 1 y todas las demás están en 0. Si hay muchos valores, se trata de amoldarlo a uno con pocos valores. Por ejemplo, los Estados de US se pueden agrupar en regiones y reducirlo de 50 a algo del orden de 5. También puede

dejarse como categorías los más frecuentes, y a los pocos ajenos dejarlos en una categoría como "Otros".

- Las variables **cíclicas** (aquellas codificadas como un número pero que son circulares, como las horas del día o un ángulo) se pueden convertir en dos variables tipo coordenadas (x,y) en un círculo.
- También podemos necesitar **discretizar** algunas variables continuas pues algunos algoritmos usan valores discretos. Pueden usarse bins del mismo ancho. El problema es que esto puede generar aglutinamiento si los datos están agrupados o tienen una escala dispersa. Otra opción es usar bins de igual altura, con lo cual los bins tendrán anchos distintos. Por lo general, es preferible usar bins de igual altura porque evita los grupos y es uniforme. También puede usarse eso como una primera distribución y después se ajustan los bins a valores informativos o más claros.
- Una cuestión importante es si hay que **normalizar** los datos para que todos estén en escalas más o menos similares, para evitar que algunas categorías dominen la distancia entre dos puntos. Una básica es normalizar todo en el intervalo $[0,1]$. A eso se lo llama normalización **min-max**. Sin embargo, esto es susceptible a outliers si tengo un mínimo o un máximo muy corridos con respecto a la mayoría de los datos. Una más usado es usar normalización **z-score**. Para eso, a un valor se lo convierte restándole la media y dividiendo por el desvío estándar.

Outliers y errores

Normalmente un valor que esté fuera de 2 veces la distancia intercuartil desde los cuartiles 1 y 3 se lo considera un outlier. Opciones para lidiar con outliers son: no hacer nada (si los valores extremos son importantes para el análisis de datos), forzar límites y acotar outliers a esos valores extremos, o hacer binning (no de igual ancho).

Preselección de variables útiles - falsos predictores

- Usualmente se remueven campos con ninguna variación (siempre) o muy poca variación (algunas veces). *Rule of thumb*: sacar las variables que toman 'casi siempre' el mismo valor. A veces, sacar variables con poca variabilidad es peligroso, ya que una pequeña diferencia puede llegar a ser muy importante para clasificar.
- Un falso predictor es un campo correlacionado con el target de mi modelado pero que no sirve para predecir. Un ejemplo típico: la nota final de un alumno predice perfectamente el estado de aprobación del curso. Hay que eliminar los falsos predictores pues nuestro modelo tiende a predecir eso en vez de la variable que nos importa. Una forma de detectar sospechos de falsos predictores puede ser por ejemplo, hacer un árbol de decisión y considerar como sospechosa a cualquier variable que identifique casi completamente a una clase al tope del árbol.

Clases desbalanceadas

En algunos casos las clases tienen muy diferentes frecuencias de aparición, como por ejemplo, el análisis de transacciones fraudulentas. Se puede tratar de cambiar la proporción de casos para llevar el desbalance a algo más soportable, sobreesampleando la clase minoritaria o subsampleando la mayoritaria. También puede usarse estrategias más inteligentes para considerar sólo las partes que sirven de la clase mayoritaria (descartando casos demasiado obvios o poco informativos)

"Si entra basura, sale basura": es fundamental preparar los datos adecuadamente para poder obtener buenos resultados.

Visualización de datos

- La visualización proporciona soporte de la exploración interactiva, dada la capacidad humana de reconocimiento de patrones y de análisis visual. También ayuda a la presentación de resultados. Como desventaja, pueden llevar a confusiones o presentar datos de manera engañosa.
- Se describe el *lie factor* como el resultado de dividir el tamaño del efecto que se ve en el gráfico por el tamaño del efecto real en los datos.
- Principios de buenas visualizaciones: darle al observador el mayor número de ideas en el menor tiempo con la menor cantidad de tinta y espacio.
- La mayoría de los métodos de visualización se usan para un primer análisis exploratorio de los datos.

Visualización en 1D

- Dot plot, con puntos en un eje
- Histograma
- Box plot
- Los outliers molestan para la representación: molestan la escala en un scatter plot, hacen bins raros en histogramas y cosas así.

Visualización en 2D

- Scatter plot. A veces se necesitan otros métodos ya que demasiados puntos tienden a saturar el gráfico, impidiendo visualizar información útil
- Gráficos de contornos, que muestran las zonas de cierta densidad. Esto es útil cuando se tienen muchos datos concentrados.

Visualización en 3D

- Gráficos de las tres dimensiones con ejes x , y , z .
- Heat maps, que hace un gráfico bidimensional y usa un código de color para visualizar la tercera variable.

Comentarios sobre gráficos para analizar datos:

- Usualmente se pueden ver histogramas o boxplots de las distintas variables una por una para analizar si hay outliers y si las distribuciones son razonables.
- Sin embargo, ver variables en múltiples dimensiones puede ayudar a detectar valores anómalos que no siguen el patrón de correlaciones entre variables.

Visualización en más dimensiones

- **Vistas múltiples:** se muestran cada variable por separado, como un histograma de cada una. Sin embargo, como dijimos antes esto no muestra correlaciones.
- **Scatterplot:** esto muestra los gráficos para todos los pares de variables. Muy útil para ver correlaciones dos a dos, pero no se ven los efectos multivariados.

- **Parallel Coordinates:** pone cada variable en un valor distinto (fijo) del eje horizontal. Los valores se ponen en la vertical de la variable correspondiente, y se unen con líneas los valores que corresponden a la misma entrada. No es tan fácil seguir patrones, pero se pueden visualizar y encontrar algunos detalles. En resumen: cada punto es una línea, si hay puntos similares tendremos líneas similares; las líneas que se cruzan muestran atributos negativamente correlacionados.
- **Chernoff Faces:** codifica las diferentes variables en características de la cara humana. Aprovecha la capacidad humana de encontrar fácilmente pequeñas diferencias entre caras.
- **Star Plot:** cada variable va en una dirección angular diferente. Cada punto forma una "constelación" usando trazos en la dirección correspondiente y con largo proporcional a su valor.

Análisis de Componentes Principales (PCA)

- Suponemos que tenemos un dataset centrado (medias 0 en sus dimensiones) y queremos encontrar una representación de esos datos en un espacio de menor dimensión. Usualmente esto sirve para poder visualizarlos, pero también hay modelos que tienen problemas de dimensionalidad y se beneficiarían de una proyección con menos dimensiones.
- Lo que caracteriza a los datos en problemas de múltiples dimensiones es la distancia entre puntos. No importa tanto cuánto valen ciertas cosas, sino tener una noción razonable de similitud de entradas. Nos gustaría una proyección lineal que preserve lo más posible la estructura de los datos (su relación de distancias)

Definición: la primer componente principal es la dirección en la cual los datos tienen máxima varianza. La segunda componente principal es la dirección en la cual los datos tienen máxima varianza, pero que es ortogonal a la primera. Y así sucesivamente.

- Una buena proyección es aquella que minimiza la diferencia entre los valores originales y sus proyecciones.
- PCA es muy útil en la detección de outliers.

Interpretación

- PCA hace una proyección lineal, un giro de los ejes.
- Cada eje principal es una combinación lineal de las variables originales.
- Se puede buscar evidencia sobre la importancia de las variables originales evaluando su aporte a las PC.
- Se suele hacer gráficamente.

Resumen

- PCA hace proyecciones lineales en bajas dimensiones que explican los datos lo mejor posible.
- Las direcciones principales son los autovectores de la matriz de covarianza.
- La importancia de cada dirección esta relacionada con su varianza explicada.

Selección de variables

Muchos problemas actuales tienen cientos o miles de variables medidas. Modelar esos problemas directamente suele ser subóptimo tanto en calidad como en interpretabilidad. Cuando tengo tantas variables, la proyección y extracción de variables puede no ser viable pues las proyecciones siguen necesitando todas las variables, con lo que no tengo una real ganancia.

Razones para seleccionar variables

- Por una cuestión de performance, pues la dimensionalidad puede ser un problema en muchos problemas de modelado, además de que muchas variables pueden agregar ruido. Nuestro modelo puede ser más fuerte teniendo menos variables. Esta razón ya no es tan relevante pues los métodos modernos son tolerantes al problema de dimensionalidad.
- Para descubrir cuáles son las variables realmente importantes en el problema, lo cual aporta descubrimientos útiles en el trabajo de dicho problema. También permite interpretar correlación, coregulación e independencia de variables.

Métodos de selección de variables

- **Univariados** (consideran de a una variable para determinar su utilidad) o **multivariados** (consideran subconjuntos de variables al mismo tiempo).
- **Filtros** (ordenan las variables de acuerdo a un criterio de importancia independientemente de la predicción) o **wrappers** (usan el predictor final para evaluar la utilidad de las variables) Los filtros suelen ser univariados mientras que los wrappers suelen ser multivariados.

Realizar una búsqueda exhaustiva en las posibles combinaciones de variables tiene dos problemas. El primero es la explosión combinatoria de los casos, lo que usualmente lo vuelve computacionalmente inviable. Aún así, un segundo inconveniente es que hacer pruebas exhaustivas puede llevar a falsos predictores en casos en las que una combinación tiene valores óptimos por sobreajuste.

Métodos de Filtro

- Elige las variables usando algún criterio de "importancia". Se puede establecer un criterio de filtro por el que las variables pueden pasar o no. En la práctica usualmente se mide una propiedad de cada variable, que suele ser un criterio distinto del que use el clasificador para conseguir dos posibles rankings. Luego, se ordena a las variables según el criterio y se retiene a las más importantes, lo cual también requiere definir un criterio de corte.
- Pueden usarse como un primer paso de selección dada su velocidad. Luego, se usa algo más robusto para hilar más fino.
- Un test paramétrico (uno que asuma la distribución de los datos) tiene mayor poder de separación en caso de que la presunción sea correcta, y es el que debería usarse si se conoce la distribución. Sin embargo, puede no dar buenos resultados si la presunción es equivocada, por lo que debería usarse un test no paramétrico en caso de que se desconozca la forma de los datos. Pasa lo mismo en la inversa, con un test no paramétrico no obteniendo distinciones tan grandes como lo haría uno específico a la distribución.

Métodos de filtro: Kruskal-Wallis, ANOVA.

Métodos de Wrappers

- Los métodos univariados no pueden resolver algunos problemas que están fuertemente relacionados a la interrelación de las variables. Múltiples variables por separado pueden ser inútiles para resolver un problema, mientras que su combinación aporta información muy útil.
- Para seleccionar las mejores variables para modelar, resuelvo el problema modelando con posibles subconjuntos y conservo la mejor solución. La búsqueda completa es exponencialmente larga, así se deben usar heurísticas de búsqueda. Búsquedas greedy:

- **Forward search:** Se comienza la búsqueda con el conjunto vacío. Luego, se construyen los posibles clasificadores que usan una variable y se conserva el que obtenga el mejor error. El proceso se repite agregando una variable cada vez. Luego, se obtiene un camino desde el clasificador sin variables hasta el que usa todas. A partir de ahí, se los compara y usa al mejor. También puede acotarse hasta cierta cantidad de variables. Una variante es la *floating search*, en la que se dan X pasos para adelante e Y para atrás (un ejemplo sería $X=2$ e $Y=1$)
- **Backward search:** Se hace lo mismo que con forward pero arrancando desde todas las variables y quitando de a una. También aplica usar floating search.

La búsqueda forward no evalúa muchas combinaciones de variables. Gran parte del camino queda condicionado a partir de las decisiones locales que se toman al principio. La búsqueda backward suele ser mejor pues considera los grupos de variables que hacen depender fuertemente la solución desde un principio. Sin embargo, no es tan útil cuando se quiere encontrar grupos pequeños y óptimos de variables.

Comparación de ambos métodos

- Tanto filtros como wrappers son heurísticas.
- Los filtros son muy rápidos, por los que son útiles para reducir problemas inmensos.
- Los filtros no resuelven el problema de modelado directamente, ni pueden analizar correctamente problemas que tengan variables fuertemente relacionadas.
- Los wrappers dan mejores selecciones al costo de ser mucho más pesados.
- A su vez, como evalúan muchos conjuntos de combinaciones de variables, suelen llevar a tomar conjuntos no óptimos, dado el overfitting.

Los wrappers backward parecerían ser potencialmente los mejores métodos de selección. Sin embargo, son computacionalmente muy pesados por la construcción de modelos y la cantidad de combinaciones que se prueban. La solución ideal sería un método backward basado directamente en el modelo final, pero más eficiente.

Métodos embebidos

- Lo que necesitamos conocer para movernos eficientemente es la derivada del error respecto de cada variable. Sin embargo, como es muy difícil conseguir eso, podríamos intentar tener alguna aproximación para usarla en cada paso. Si la función error es razonablemente suave, dar el paso en la dirección del máximo descenso de la derivada debería ser lo mismo que el máximo descenso del error.

Recursive Feature Elimination (RFE):

- Se basa en ajustar un modelo a los datos y rankear las variables usando una medida interna de importancia, siendo más importante la que empeora al modelo al ser eliminada. Cada paso consiste en construir mi modelo, ordenar mi conjunto de variables usándolo, eliminar la menos importante y repetir ese paso hasta no tener variables.
- Medidas de importancia: SVM, Random Forest, LDA, PDA, etc.
- Para SVM, un método para conocer la importancia de las variables es conseguir el vector perpendicular al hiperplano de separación y luego medir las magnitudes de cada componente del mismo. Las variables que tienen un mayor valor son más importantes para separar que las que tienen menores valores.
- Es importante tener las variables escaladas en RFE para que tengan sentido las medidas usuales de relevancia de cada una.

- Un problema con RFE ocurre cuando tenemos múltiples variables importantes pero correlacionadas. Cuando la interrelación entre N variables divide bien, ellas "comparten" la importancia, con lo cual todas son menos importantes de lo que deberían ser. Entonces, cuál se elimina y cuál se usa a veces termina siendo chance, lo que vuelve el proceso un poco inestable.
- Los métodos embebidos son rápidos, efectivos, entendibles y más estables que los wrappers greedy. Sin embargo, la importancia de variables se estima en vez de medirse directamente, sumado a problemas de inestabilidad con variables correlacionadas.

Evaluar las selecciones

- Resulta útil tener idea de cómo cambia el error de predicción de mi modelo al eliminar variables. La idea base consiste en aplicar algún método de selección al problema y controlar el error mientras se van eliminando variables. Luego, usamos el mínimo. En un principio, se podría usar cross-validation para determinar el error. El problema con esto es que se estarían usando los mismos datos para medir el error de predicción así como para elegir la validez de las clases. "Elegir variables es como construir el clasificador" ya que guía el proceso de búsqueda, con lo cual es importante usar otro conjunto de variables para poder tener estimaciones correctas durante el proceso.
- Procedimiento para evaluar las selecciones: partir los datos en training, validación y test. Para cada subconjunto de variables se entrena un modelo en el training set. Luego, se toma el subconjunto que muestra la mejor performance en el validation set. Para mejorar esta estimación, se puede usar cross validation entre estos dos sets. Luego, puede medirse el error verdadero en el test set.

Clustering

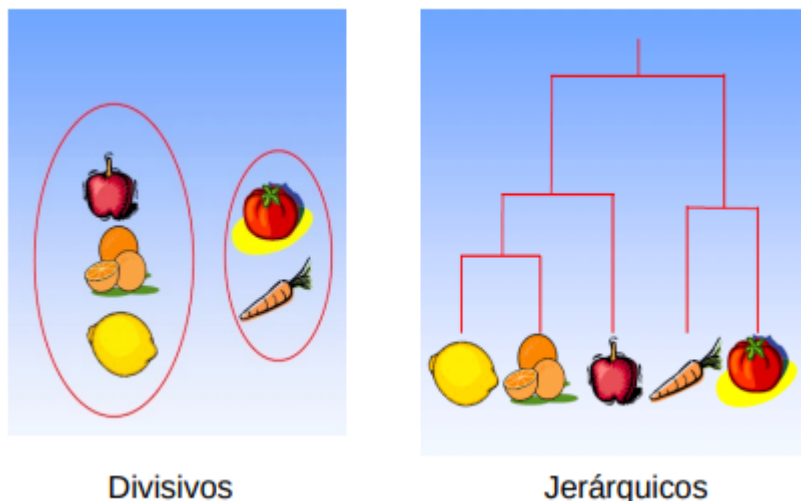
Un cluster es un agrupamiento de objetos. Definimos conceptualmente como grupo a un conjunto de objetos que son muy similares entre sí pero diferente de los demás. Esto requiere una métrica que nos dé una idea de similitud entre objetos.

La idea intuitiva del problema de clustering refiere a, dados un conjunto de datos y una métrica, encontrar una partición de los objetos tal que los que estén en el mismo grupo son similares, y los demás son distintos a ellos.

El objetivo de esta práctica es principalmente descubrir información sobre los datos. Es decir, resulta interesante encontrar "grupos naturales" en un conjunto de datos del que no se conocen las clases, además de encontrar jerarquías de similaridad en los datos (taxonomía). También permite resumir los datos al encontrar "prototipos" que sean representativos de un conjunto grande de ejemplos.

Clustering no es una práctica de clasificación. Es un aprendizaje no supervisado en el que se conocen los datos pero no los grupos en que están organizados. El objetivo es encontrar la organización. Es decir, clasificación se centra en encontrar una regla que separe los datos con las clases que ya conozco, mientras que clustering trata de asignar etiquetas a los grupos que los divida razonablemente.

Hay dos clases de algoritmos de clustering: los **divisivos**, que consideran el clustering como una partición del espacio y busca la partición más significativa en un número fijo de subconjuntos, mientras que el **jerárquico** busca una anidación de particiones de la que luego puede extraerse una cantidad dada de partes.



Las representaciones usuales de los datos para clustering vienen en dos formas: los datos vectoriales en una matriz o una matriz de distancias. En el primero tenemos dos modos de los datos (filas y columnas), mientras que en el segundo tenemos un único modo (se usan las variables tanto para filas como columnas).

Las métricas pueden definirse de muchas formas. Para datos binarios, ordinales o categóricos se pueden definir medidas especiales. En general, es razonable que sean siempre positivas y que respeten la desigualdad triangular.

Asignarle distintos pesos (escalas) a las variables de los datos permite que algunas dominen la métrica, lo cual nos puede dar resultados muy distintos.

El algoritmo de clustering general consiste en definir una métrica para poder calcular todas las distancias entre datos. Luego, se define una medida de bondad del clustering. Por último, se minimiza la medida de bondad, normalmente con alguna heurística.

Clustering divisivo - K-means

El algoritmo más viejo y base para estos métodos es K-means. El objetivo es encontrar una partición de los datos en k grupos tal que la distancia media dentro de los puntos de cada grupo sea mínima. Se buscan clusters compactos, con lo que al minimizar la distancia dentro del grupo se maximiza la distancia entre grupos.

Una noción similar a buscar minimizar la distancia media entre los puntos del cluster es buscar que se minimice la suma de distancias al centro del cluster, donde el centro de un cluster es el vector medio de todos los puntos que lo componen. Esta segunda noción es mucho más simple de resolver que la primera, por lo que la adoptaremos de ahora en adelante.

La función de costo total depende de dos factores: qué centro de cluster se le asigna a cada punto y la distancia que tiene dicho punto al centro correspondiente. Es sencillo notar que la asignación que minimiza el costo total es la que a cada punto le da el centro más cercano.

De otra manera, si los clusters ya están dados, determinamos que el centro que minimiza la suma de distancias es el punto medio de los vectores (el centroide).

Algoritmo base: un proceso para poder realizar la minimización de la función de costo consiste en una minimización alternada. Primero, defino un conjunto de centros al azar (usualmente buscando mínimo y

máximo de cada variable). Luego, etiqueto los datos con respecto al centro del cluster que tienen más cercano. Una vez que tengo eso, redefino las posiciones de cada centro en función de los puntos de su cluster y repito el procedimiento. Se puede demostrar que iterar este procedimiento hace que la suma de distancias descienda siempre. Esta iteración tiende a un mínimo local de la función en tiempo finito.

Los criterios de parada involucran llegar a un punto en el que no hay cambios o que se tiene un cambio cíclico en un conjunto mínimo de puntos.

Este método es lineal en el número de entradas y que tiene garantía de convergencia a un mínimo local.

Sin embargo, esta condición de que caiga en mínimos locales es su mayor problema. Una idea para solucionar esto es realizar varias corridas en múltiples lugares y se queda con la mejor. Notar que en esta solución no hay overfitting, pues no voy a generalizar nada a partir de estos datos sino que justamente estoy buscando la solución más apretada para estos datos específicamente. Es decir, no hay problema en hacer mil corridas y quedarme con la mejor.

Su segundo problema es que depende fuertemente de los outliers. Una posible modificación es usar una medida más resistente a los outliers, como usar la mediana de las distancias en vez de la distancia media.

Un tercer problema es que sólo vale en espacios vectoriales. El método se basa en recorrer el espacio con las posiciones de los puntos. Si tuviéramos sólo las distancias entre puntos, no podemos resolver el problema con esta implementación de K-means.

K-medoids

Este método combate los problemas que mencionamos de K-means. Se basa en representar a cada cluster por su medoid, que es el centro geométrico de la distribución. Es decir, es el punto de la distribución que esté más cerca de todos los demás a la vez. Por ende, estamos usando un punto de los datos y no uno inexistente. La iteración se realiza de una manera similar que en K-means.

De esta forma, se soluciona el problema de los outliers y vale para espacios arbitrarios, no necesariamente vectoriales. El problema de los mínimos locales se mantiene pues el método de búsqueda es el mismo.

El precio que se paga en este algoritmo es que ahora tiene una complejidad cuadrática con respecto a la cantidad de entradas en los datos.

Clustering jerárquico

Hay problemas con jerarquías naturales en los datos, como las taxonomías naturales. También puede resultar interesante encontrar una estructura en niveles en los datos.

En este tipo de métodos se busca organizar los datos en una serie de particiones anidadas. En cada nivel, los datos son más similares entre sí que si se los compara con los de otros grupos.

La representación usual son los dendogramas: diagramas de divisiones que representan las categorías basándose en el grado de similitud. Las líneas verticales representan el grado de similitud de las clases que se muestran horizontalmente. Otro tipo de representación es usando un diagrama de Venn, aunque son menos utilizados.

El problema de encontrar estos clusters tiene dos posibles estrategias:

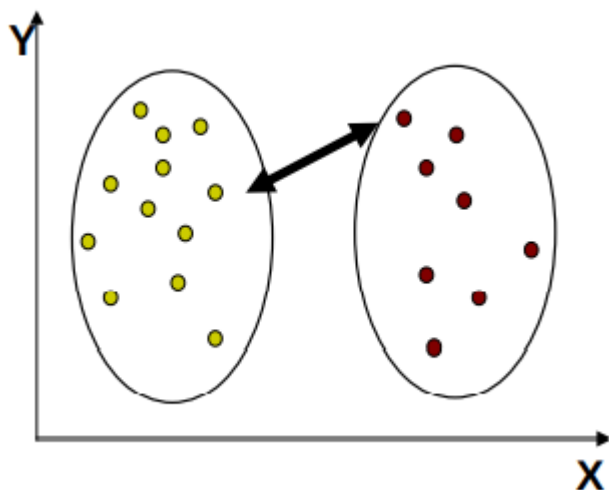
- **Top-Down o Divisiva.** Es muy poco usada. Comienza con todos los puntos en un cluster único y en cada paso divide todos los clusters en dos partes de acuerdo a un criterio a optimizar. Eventualmente termina con todos los puntos separados.
- **Bottom-Up o Aglomerativa.** Realiza el proceso inverso, comenzando con todos los puntos separados y uniendo los dos clusters más similares según algún criterio en cada paso.

Para realizar un método divisivo, en cierta forma se requiere un método que me permita hacer clusters grandes, entonces estaría necesitando otro método que me realice un clustering de los datos que estoy dividiendo. Sin embargo, en el caso de necesitar pocos clusters de muchos datos, puede resultar útil.

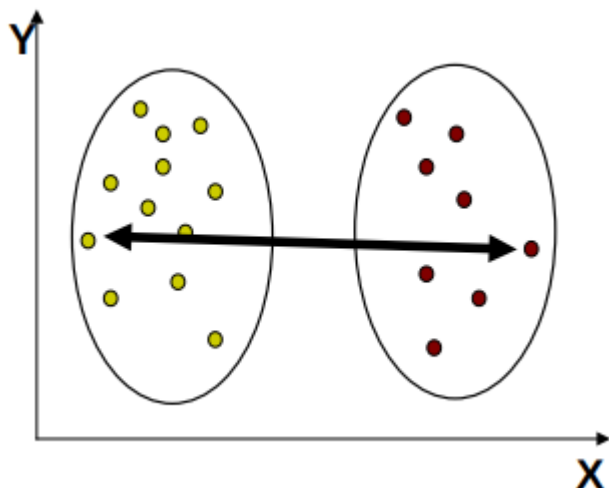
Por otro lado, el método aglomerativo tiene un procedimiento más natural y sólo requiere una noción de distancia para poder hacerse. Sin embargo, necesitamos una noción de similitud entre clusters, no entre puntos.

Medidas de similitud más comunes:

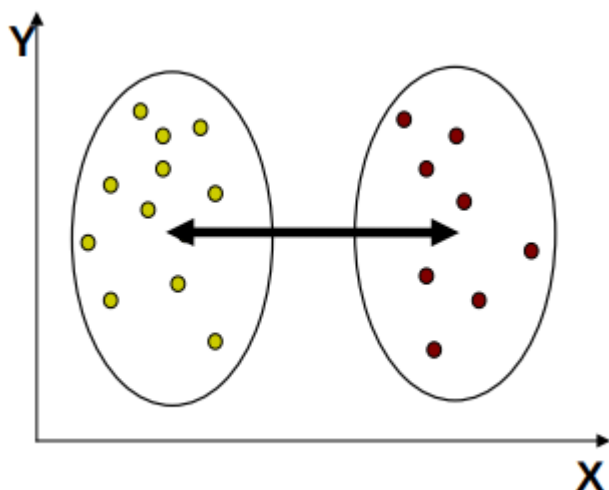
- **Single Linkage:** La medida de distancia entre dos clusters es la mínima distancia entre todos los pares de puntos que pertenecen a clusters distintos. Este método busca una alta conectividad, no grupos compactos. Entonces, se encuentra una continuidad espacial más que grupos apretados. Entonces, en cierta forma este método construye un minimum spanning tree de los datos usando el algoritmo de Prim. Este método es muy dependiente de outliers, con lo que no se pueden encontrar pocos clusters muy poblados, sino que usualmente los últimos valores que se unen en el top level son outliers. También es ineficiente al tener que comparar todos los puntos entre sí múltiples veces. Además, se construye sobre errores previos y no revisa asignaciones anteriores con lo que no se puede corregir.



- **Complete Linkage:** Acá se toma una noción inversa, pues se toma como medida de distancia la distancia máxima entre todos los pares de puntos que pertenecen a dos clusters diferentes. Esto agrupa "conjuntos completamente vecinos", por lo que resulta en grupos compactos, estilo K-means. Es dependiente de outliers (de una forma diferente). Es parecido a K-means pero es determinista. Sin embargo, es ineficiente y no corrige errores previos como como Single Linkage, además de no poder formar clusters con formas arbitrarias.



- **Average Linkage:** Usa la medida promedio entre todos los pares de puntos de clusters diferentes. Da una solución intermedia a la de los dos anteriores, buscando grupos tanto conectados como compactos. Da soluciones distintas a K-means.



- **Criterio de Ward:** Busca los clusters que al juntarlos dan el menor aumento en la suma de distancia cuadrada entre sus componentes. Usualmente da resultados equivalentes a K-means, pero da toda la jerarquía. Si queremos una cantidad pequeña de clusters, es preferible usar K-means directamente.

Resumen de clustering

En la gran mayoría de las aplicaciones se usa alguno de ambos métodos. Ambos son heurísticas simples sin gran teoría por detrás. Resultan fáciles de implementar y en la práctica los dos dan resultados razonables muchas veces. A su vez, sirven de base para métodos más elaborados.

"En clustering, uno encuentra lo que busca, no necesariamente lo que hay." Es decir, uno siempre va a encontrar la mejor solución para lo que busca la heurística utilizada (por ejemplo, los grupos más compactos o los grupos con mayor continuidad espacial). Cada método de clustering parte de imponer una estructura, encontrando lo mejor posible a lo que estén buscando. Si la premisa del método es adecuada para el problema, en general el resultado es bueno. No existen algoritmos que puedan resolver dos "clases de problemas" diferentes de manera correcta.

Estabilidad

El problema de estabilidad radica en buscar la cantidad de clusters que deben usarse en un problema. En algunos problemas la cantidad es fija y está dada en el problema, pero en la mayoría no pues estamos estudiando si hay una estructura que los aglomere. Lamentablemente, los algoritmos de clustering siempre devuelven una solución aunque no tenga sentido. El número de clusters es otra información que deberíamos extraer de los datos.

La cantidad de clusters en un dataset es una pregunta difícil pues no hay "verdad" contra la que comparar y los métodos no poseen una fuerte teoría detrás. Por ende, la mayoría de los métodos son empíricos y no presentan reales garantías pero han sido probados útiles en la práctica.

Método del salto

Un criterio general para evaluar la calidad de la solución es la suma de las distancias dentro de cada cluster. Eso es lo que busca minimizarse en la mayoría de los métodos. Sin embargo, agregar clusters siempre se reduce la suma de las distancias, por lo que no podemos buscar un mínimo. Más allá de eso, decrece más cuando se separan dos clusters verdaderos que si divide en dos un "cluster natural". Entonces, hay que buscar un cambio repentino de pendiente en la gráfica de la suma de distancias en función del número de clusters. Este método no suele ser una medida confiable usada directamente, además de que no detecta la ausencia de clusters.

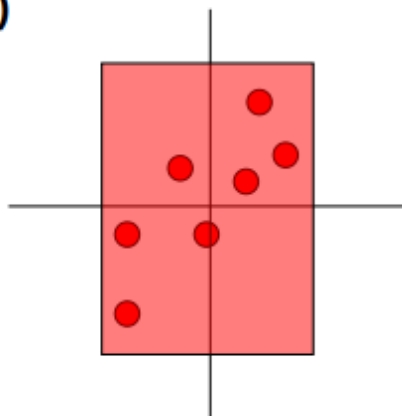
Gap Statistic

Intenta resolver los problemas del método del salto. La idea consiste en comparar la curva anterior con la curva que da una distribución uniforme. Se busca cuantificar el salto al buscar la primera diferencia significativa entre las dos curvas, la del problema que se está estudiando con el salto que da en el mismo caso una distribución uniforme. Si se diferencian adecuadamente, determinamos que hay clusters. Se busca la primera cantidad de clusters en la que hay una diferencia significativa.

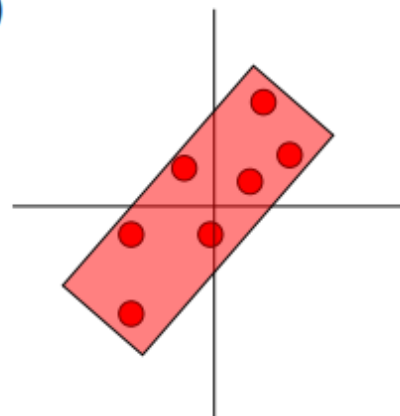
Para generar la referencia contra la que compararemos, hay dos formas:

- Tomar una distribución uniforme que ocupe el mismo hiper-rectángulo que la original.
- Hacer lo mismo pero sobre un PCA de los datos.

a)



b)



Método de la estabilidad

La idea base es que los resultados científicos tienen que ser reproducibles. Los "clusters naturales" de un problema se tienen que encontrar siempre. Si cambiamos un punto del problema y la solución desaparece, entonces no son clusters naturales sino un resultado ficticio creado por el algoritmo.

La idea base del algoritmo es, variando la cantidad de clusters, construir muchas soluciones replicadas (leves modificaciones en los datos), evaluar la estabilidad de las soluciones y seleccionar el k con mayor estabilidad.

Como muchos algoritmos son deterministas, es necesario generar soluciones replicadas que generen problemas perturbados. Sin embargo, hay un trade-off al perturbar: si cambiamos mucho podemos destruir la estructura natural, mientras que si cambiamos poco puede parecer una solución estable aunque no lo sea.

Métodos para generar réplicas

- **Subsample:** tomar una muestra al azar de los datos originales. Usualmente se toma entre el 70% y el 90%.
- **Agregar ruido:** se agrega ruido blanco (normal) a los datos originales con un bajo porcentaje de la señal (menos del 10%)
- **Proyecciones:** en datos de alta dimensionalidad, tomar proyecciones sobre un subespacio elegido al azar. Una vez que los datos están perturbados, se usa siempre el mismo algoritmo, buscando que la única variación sean los datos. Luego, debe medirse cuán diferentes son las soluciones.

Cuando se evalúan dos soluciones que usen los mismos datos, existen distintas fórmulas que realizan esto, pero la noción básica en los métodos es considerar si cada par de puntos que están en el mismo cluster en una solución permanecen en la otra. Cuando se evalúa sobre conjuntos que tienen datos distintos, hay dos opciones:

- **Contraer:** evaluar solamente sobre la intersección de ambos conjuntos, que tiene sentido si la muestra es grande, pues las pérdidas de información se vuelven más relevantes. Esta solución es más sencilla y más utilizada.
- **Extender:** se agregan los puntos faltantes a cada solución obtenida previamente. Por ejemplo, para k-means se asigna el centro más cercano, mientras que en Single Linkage asigno el cluster de su primer vecino espacial.

Luego, tenemos una muestra de los valores de similaridad entre las soluciones para distintos k. La evaluación más simple es quedarse con el que tenga la media máxima de estabilidad, pero no es necesariamente el valor más correcto. Una segunda opción es mirar la concentración del histograma de las distribuciones. Lo que se propone para esta idea es quedarse con la solución estable con la mayor cantidad de clusters posibles.

Este método asume que las soluciones naturales tienen que ser estables. El problema es que lo contrario no está garantizado, algunas soluciones artificiales pueden ser estables también.

Resumen de Estabilidad

Gap y el método de estabilidad son dos heurísticas muy usadas. La primera compara la bondad de la solución frente a una distribución nula, mientras que la segunda asume que las soluciones naturales tienen que ser estables. Aún así, ambos métodos son útiles para determinar la ausencia de clusters.

Métodos de ensambles

Ensamble: conjunto grande de modelos que se usan juntos como un 'meta modelo'.

La idea es usar el conocimiento de distintas fuentes al tomar decisiones.

Tenemos dos **componentes base**:

- Un método para seleccionar o construir los miembros.
- Un método para combinar las decisiones.

Con respecto a los ensambles tenemos:

- **Ensamblés divisivos:** dividen el problema en una serie de subproblemas con mínima sobreposición. Es una estrategia del tipo 'divide and conquer'. Son útiles para atacar problemas grandes. Necesitamos una función que decida que clasificador tiene que actuar.
- **Ensamblés planos:** se basan en la idea de tener 'muchos expertos, todos buenos'. Se necesita que sean lo mejor posible individualmente ya que sino por lo general no sirven. Pero también necesitamos que opinen distinto en algunos casos, si todos opinan igual me conviene quedarme con uno solo.

Se trata de formalizar esto vía la estadística. Se habla del **dilema sesgo-varianza**: los predictores sin sesgo tienen alta varianza (y viceversa). Dos formas de resolver esto son:

- Disminuir la varianza de los predictores sin sesgo. Para esto se pueden construir muchos predictores y promediarlos. Ejemplo: Bagging y Random Forest.
- Reducir el sesgo de los predictores estables. Podemos construir una secuencia tal que la combinación tenga menos sesgo. Ejemplo: Boosting.

Ensamblés Divisivos

Bagging

Bootstrap Aggregating. Consiste en usar predictores buenos pero inestables como árboles y redes neuronales. Luego se trata de perturbar los datos para conseguir diversidad: como los predictores son inestables, pequeños cambios en los datos, dan cambios importantes en los modelos.

Reduce la varianza al promediar predictores diversos.

La idea de bagging es una *bootstraps*. Es una muestra al azar de la misma longitud, con repetición:

```
sample(1:10, rep=T)
> 3 5 4 3 1 7 9 9 2 1
```

No introduce bias en ninguna estadística. Luego, cada predictor se entrena sobre un bootstrap del conjunto de entrenamiento.

Regla de combinación:

Para regresión/ranking se hace el promedio simple de las predicciones de cada modelo.

Para clasificación tenemos dos opciones:

- Se hace el voto mayoritario.
- Cada modelo estima la probabilidad de la clase. Se promedian y se elige la de mayor probabilidad.

Para Clustering creamos una matriz que cuenta cuantas veces cada par de puntos termina en el mismo cluster, y clusterizamos esa matriz de similitud.

Variantes para perturbar los datos son:

- Subsampling (bastante similar al bootstrapping).
- Agregar ruido.
- Variar los modelos en vez de los datos: modificar los parámetros de aprendizaje para los métodos.

Otras formas de combinar:

- Usar probabilidades en vez de votos.
- Usar pesos estadísticos en la combinación:
 - Los mejores clasificadores tienen más fuerza en la decisión. Funciona igual o mejor que el uniforme.

Random Forest

Surge como una mejora de bagging para árboles. El problema que tiene el bagging con los árboles es que usar bootstraps genera diversidad pero los árboles siguen estando muy correlacionados, entonces las mismas variables tienden a ocupar los primeros cortes siempre.

La solución que se propone es agregar un poco de azar: en cada nodo se elige un subconjunto de variables al azar y evaluamos solo esas, observemos que:

- No agrega sesgo: a la larga todas las variables entran en juego.
- Agrega varianza pero se soluciona fácilmente promediando modelos.
- Es efectivo para decorrelacionar los árboles.

Este método construye los árboles hasta separar todo, sin podado ni criterio de parada. El número de árboles no es importante mientras sean muchos.

Suele tener mejores predicciones de bagging. Es muy usado ya que es casi automático. Los resultados son comparables a los mejores métodos actuales.

Subproductos de RF:

- **Out-of-bag estimation:** cuando se toma un bootstrap hay puntos OOB que quedan afuera. Para cada predictor hay un conjunto OOB que no se usó durante el entrenamiento. Entonces, podemos usar esos para estimar errores de predicción sin sesgo. Son una buena estimación del error de test ya que al estar calculadas con un ensamble más chico, estiman de más, por lo que tienen un sesgo 'seguro'.
- **Medida de importancia de la variables:** tenemos dos criterios posibles para esto:
 - *Gini:* calcula un índice de utilidad de cada variable en las divisiones y lo promedia entre todos los árboles (las variables útiles reducen el Gini mucho más).
 - *Randomization:* convierte una variable en ruido al desordenarla completamente, y se promedia cuánto modifica las predicciones del ensamble usando los datos OOB.

Los dos métodos se estiman fácilmente y suelen tener resultados equivalentes.

- **Gráfica de proximidad:** es un modo interno de este algoritmo de proyectar datos en bajas dimensiones (como PCA). Lo hace calculando una matriz de distancias entre los datos basada en cuántas veces los puntos terminan en el mismo nodo de un árbol, considerando que puntos cercanos deberían terminar juntos más seguido. Luego, hace un MDS de esas distancias.

Boosting

La pregunta fundamental para este modelo es si es posible conseguir una solución de error arbitrariamente chico en cualquier problema (*stronglearner*) como una combinación de varios modelos que sean apenas mejores que elegir al azar (*weaklearner*). Entonces, la idea central es contruir una secuencia de clasificadores débiles, donde cada uno aprenda un poco de lo que le falta a la secuencia previa. Como los clasificadores débiles pueden mejorar un poco en cualquier problema, la secuencia converge a error cero.

Para los conjuntos de entrenamiento, se trabaja con bootstraps, como en bagging. Sin embargo, en la muestra no se toma con pesos estadísticos uniformes. Se busca poner énfasis en aprender los puntos que fueron mal clasificados previamente por el ensamble. Para eso, se le aumenta el peso estadístico a los errores, y se le disminuye a los aciertos.

Para combinar los votos de los clasificadores, se hace una suma de votos, como en Bagging. Sin embargo, en dicha suma se le da más peso a los clasificadores que son mejores.

Se puede probar que el error sobre los datos de ajuste converge a cero si el weaklearn de verdad puede siempre hacer algo mejor que el azar. Sin embargo, aunque el error de entrenamiento tienda a cero, el clasificador se va volviendo más complejo a cada paso, con lo cual se espera que esto sobreajuste. Aún así, el modelo en general no sobreajusta. El error de test incluso sigue bajando aunque el error de entrenamiento ya esté en 0.

Una posible explicación de ese fenómeno viene de la teoría del margen. El error en test sólo mide si se acertó con la clase, pero lo que realiza el algoritmo es más complejo, pues es la suma pesada de votos por una clase y la otra. Por ende, hay que tener en cuenta la "fuerza" de cada respuesta. Para eso se define el margen de cada decisión, que es la fracción pesada de votos correctos menos la fracción incorrecta de las decisiones. Luego, se demostró que un mayor margen es equivalente a una menor cota superior en el error de test y que boosting incrementa en cada iteración el margen de los datos. Por ende, el error en test baja porque boosting aumenta el margen.

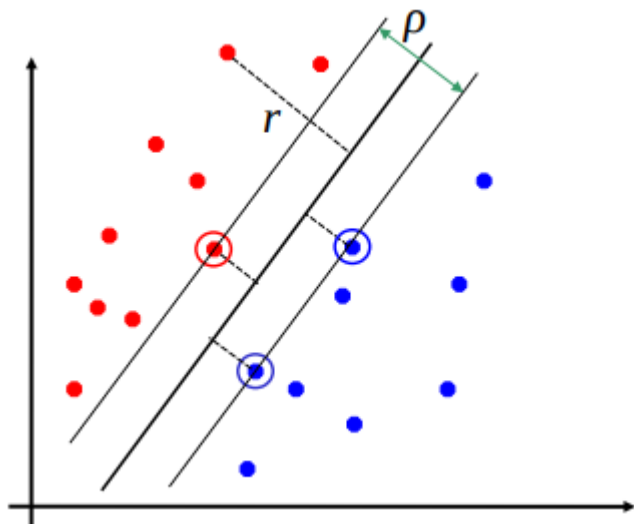
Otros enfoques explican de maneras diferentes el mismo proceso: Boosting logra disminuir la cota superior del error en test conforme sigue iterando.

Boosting tiene el mejor funcionamiento para clasificación binaria, pero se lo ha extendido (aunque sin resultados tan buenos en la práctica) para regresión, clasificación múltiple y ranking. Boosting tiene resultados de orden similar a Bagging, pero para redes neuronales no se desempeña tan bien. Esto se debe a que las redes son demasiado flexibles para el requerimiento de boosting de tener clasificadores rígidos.

Métodos de Kernel

SVM

Supongamos que tenemos un hiperplano que separa dos clases:

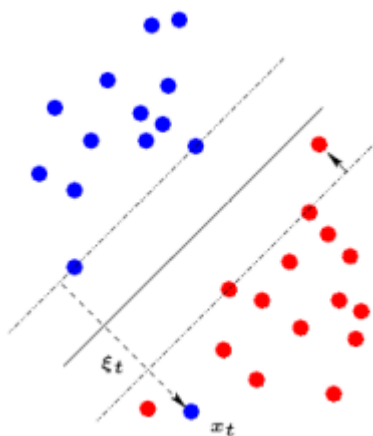


Los ejemplos más cercanos al hiperplano se los llama **support vectors**. El **margen** ρ del separador es la distancia entre los support vectors.

El objetivo es **maximizar ese margen**. Esto implica que **sólo importan los support vectors** y que el resto de los ejemplos de entrenamiento se pueden ignorar. La idea es **poner el hiperplano lo mas lejos posible de ambas clases**, para que al tener que clasificar un nuevo ejemplo, se pueda hacer con mayor seguridad.

Entonces la idea de SVM es: **dado un conjunto de entrenamiento, encontrar un hiperplano que separe ambas clases y maximice el margen**. Mientras más grande sea el margen, mas pequeño será el error de test.

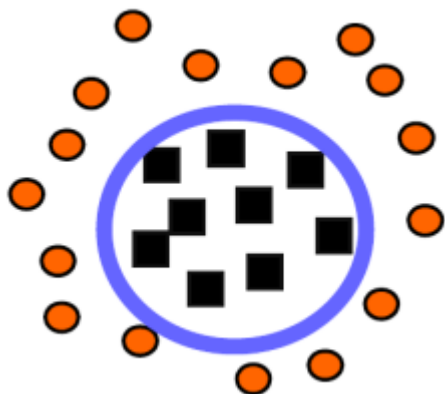
El problema es que si las clases no son separables el problema no tiene solución.



Entonces se puede usar un **margen suave**. Se agregan lo que se llaman 'variables de relajación' que permiten que para algunos puntos el problema sea más fácil. Esto lo que hace es **permitir que algunos ejemplos sean mal clasificados pero trata de minimizarlos** para que estos sean la menor cantidad posible. El Lagrangiano nos ayuda en esta cuestión.

Kernels

Puede haber problemas en los que por más que SVM encuentre una solución, esta no sea muy buena:



Proyectan los datos en un espacio de una dimension más alta, donde debería ser más fácil encontrar un hiperplano que separe las clases. La idea es que, como ya vimos, al pasar a más dimensiones las cosas se tienden a separar, entonces por eso debería ser más simple encontrar dicho hiperplano.

El problema que tiene es que se tienen que calcular los productos para las variables proyectadas y esto en un espacio de dimensión muy grande puede llegar a ser muy costoso.

Entonces se usan las funciones *kernel* que representan un producto escalar dado. Esto permite que no se tenga que calcular el producto escalar sino que se pueda calcular directamente la función kernel. Estas funciones tienen ciertas propiedades matemáticas que permiten que esto sea posible. Entonces nosotros buscamos funciones de kernel que sean útiles. Entre ellas tenemos:

- **Polinomiales:** tienen la forma $k(x, z) = (uxz + v)^p$ y se trata de optimizar p .
- **Gaussianas:** tienen la forma $k(x, z) = \exp(-\frac{\|x - z\|^2}{2\sigma^2})$. Se trata de optimizar σ .

Cualquier kernel positivo semidefinido corresponde a un producto en algún espacio cualquiera. Por lo que hacer combinaciones lineales de kernels conocidos, nos permite crear nuevos kernels para aplicar a nuestro problema.

Resumen SVM y Kernel

- En la práctica, para usar SVM primero se elige la función de kernel que se va a usar. Luego se optimiza el parámetro de la función de kernel. Finalmente, se optimiza el parámetro C , el trade off entre el margen y los errores. Para datasets sin ruido, este parámetro no tiene mucha influencia, pero para datasets con ruido debe ser elegido cuidadosamente; valores pequeños dan por lo general mejores resultados.
- Las SVM dan una solución única que siempre se encuentra y es óptima.
- Usamos un conjunto de validación para optimizar los parámetros del kernel.

